Principes SOLID – TP1 – Travailler sur du code de mauvaise qualité

Nous allons améliorer sur une application permettant la gestion des documents de projets étudiants. Le code source de cette application est disponible sur Moodle. Nous allons y ajouter de nouvelles fonctionnalités.

Introduction

1. Créez un nouveau projet Eclipse et importez le code source. Exécutez la classe `Main` pour s'assurer du bon fonctionnement et générer les données fictives.
2. Prenez connaissance du code existant :
   (a) A quoi servent les classes `Etudiant`, `Formation` et `Projet` ?
   (b) Tracez un diagramme de classe UML représentant les relations entre ces classes.
   (c) Que contient le paquetage `document` ? Ajoutez ce paquetage à votre diagramme de classe.
   (d) Que contient le paquetage `document.compterendu` ? Ajoutez ce paquetage à votre diagramme de classe UML.
   (e) Que contient le paquetage `document.export` ? Ajoutez ce paquetage à votre diagramme de classe UML.
   (f) Que contient le paquetage `util` ? Ajoutez ce paquetage à votre diagramme de classe UML.
3. Rappelez les cinq principes SOLID.
4. Dans le code source, indiquez les principes SOLID qui ne sont pas respectés (précisez où et pourquoi).

Nous allons maintenant ajouter de nouvelles fonctionnalités. Cependant, afin de conserver la compatibilité, nous ne pouvons pas modifier la structure du code – nous sommes donc obligés de mal coder, et la dette technique va s'accumuler.
Téléchargez la classe `Main2` sur Moodle et ajoutez-la dans le paquetage `tp_solid`. L'objectif des questions ci-dessous est de proposer une nouvelle version telle que `Main` et `Main2` fonctionnent toutes les deux (ce qui « garantit » la compatibilité avec la version précédente).

1. Permettre la gestion des projets des étudiants en InfoCom

On souhaite pouvoir utiliser l'application pour le parcours InfoCom.

1. Dans la classe `Main2`, la formation de l'étudiant est changée en `"INFOCOM"` (ligne 16), bien que la formation ne soit pas codée. Cela lève-t-il une erreur ? Quand ?
2. Ajoutez le parcours InfoCom (les informations nécessaires sont disponibles sur le site de l'IUT).
3. Quelles attributs, méthodes et commentaires avez-vous dû modifier ? Auriez-vous pu le faire si vous disposiez uniquement du code compilé (.jar) ?

2. Permettre l'affichage des plannings en XML

On souhaite permettre l'affichage en XML, sous la forme suivante.

```
<planning>
  <etudiant nom="Nomde Létudiant" />
  <item date="10/04/2025" projet="Projet Web" />
  <item date="11/04/2025" projet="Archical" />
  <item date="15/04/2025" projet="Archical" />
  <item date="16/04/2025" projet="Projet Web" />
</planning>
```

1. Ajoutez la méthode `afficherPlanning(String format)` à la classe `Etudiant`, où l'argument `format` peut prendre les valeurs `txt` ou `xml`.
2. Critiquez la solution apportée.
3. Quelles attributs, méthodes et commentaires avez-vous dû modifier ? Auriez-vous pu le faire si vous disposiez uniquement du code compilé (.jar) ?
4. On souhaite maintenant pouvoir choisir le format de date français `"dd/MM/yyyy"` ou américain `"MM/dd/yyyy"`. Proposer une extension pour pouvoir choisir le format de date lors de l'affichage "txt" et "xml".

3. Quelques tests sur les fichiers

On souhaite pouvoir apposer à la fin de chaque document une signature. Les méthodes `signerDocument` et `estSigne` sont présentes dans la classe `Main2`.

1. Vérifiez que les documents du projet1 ne sont pas signés :

$$\texttt{doc.open}(); \implies \texttt{estSigne}(\texttt{doc}, \texttt{sign}) = \texttt{FALSE}.$$

2. Vérifiez que la méthode `signerDocument` respecte la spécification suivante :

$$\begin{aligned} &\texttt{doc.open}();\\ &\texttt{signerDocument}(\texttt{doc}, \texttt{sign}); \implies \texttt{estSigne}(\texttt{doc}, \texttt{sign}) = \texttt{TRUE}. \end{aligned}$$

3. Vérifiez que la méthode `document.open()` écrase les modifications :

$$\begin{aligned} &\texttt{doc.open}();\\ &\texttt{signerDocument}(\texttt{doc}, \texttt{sign});\\ &\texttt{doc.open}(); \implies \texttt{estSigne}(\texttt{doc}, \texttt{sign}) = \texttt{FALSE}. \end{aligned}$$

4. Vérifiez que la méthode `document.save()` enregistre les modifications :

$$\begin{aligned} &\texttt{doc.open}();\\ &\texttt{signerDocument}(\texttt{doc}, \texttt{sign});\\ &\texttt{doc.save}();\\ &\texttt{doc.open}(); \implies \texttt{estSigne}(\texttt{doc}, \texttt{sign}) = \texttt{TRUE}. \end{aligned}$$

5. Critiquez vos résultats.

4. Export des comptes-rendus

La classe `Projet` offre la méthode `exportDocuments` qui permet d'enregistrer tous les documents du projet dans un fichier XML, grâce à son attribut `ExportXML`. On souhaite pouvoir exporter aussi les comptes-rendus.

1. Ajoutez à la classe `Projet` la méthode `exporterComptesRendus` qui exporte les textes et diaporamas de tous les comptes-rendus sur un principe similaire.
2. Critiquez la solution apportée.
3. Quelles attributs, méthodes et commentaires avez-vous dû modifier ? Auriez-vous pu le faire si vous disposiez uniquement du code compilé (.jar) ?

5. Export en mode TXT

On souhaite pouvoir exporter les documents et les comptes-rendus en utilisant `ExportTXT` ou `ExportXML`.

1. Tracez le diagramme de classe du paquetage `document.export`.
2. Proposez une solution pour utiliser `ExportTXT` qui maintient la compatibilité avec la version précédente.
3. Critiquez la solution apportée.
4. Quelles attributs, méthodes et commentaires avez-vous dû modifier ? Auriez-vous pu le faire si vous disposiez uniquement du code compilé (.jar) ?